An Example Implementation of RiTS

Rohit Kanrar

2025-06-29

Small Scale Implementation of Simulation Experiment in Section 3

Code from the two following files are relevant for this section:
- `code/simulation/replicated_trial.R` - `code/simulation/replicated_trial_evaluation.R`

Trial Optimization

Loading Necessary Functions, Objects and Simulated Data

```
source("code/function/main_function.R")
source("code/function/misc.R")
source("metadata/get_metadata.R")
sim_choice <- readRDS("metadata/sim_choice.RData")
sim_dat <- readRDS("metadata/sim_dat.RData")
dgps <- c("low", "high")
```

Creating a Total of 24 Cases For Choices of Hyper-parameters and DGPs

Please refer to Web Tables 1 and 3 for more details.

```
tr_starts <- sim_choice$tr_start
min_prpns <- sim_choice$min_prpns
cases <- expand.grid(dgp = dgps, min_prpn = min_prpns, tr_start = tr_starts)
print(cases)
```

```
##     dgp min_prpn tr_start
## 1   low    0.005       24
## 2  high    0.005       24
## 3   low    0.010       24
## 4  high    0.010       24
## 5   low    0.050       24
## 6  high    0.050       24
## 7   low    0.100       24
## 8  high    0.100       24
## 9   low    0.005       40
## 10 high    0.005       40
## 11  low    0.010       40
## 12 high    0.010       40
## 13  low    0.050       40
## 14 high    0.050       40
## 15  low    0.100       40
## 16 high    0.100       40
## 17  low    0.005       56
```

```
## 18 high    0.005       56
## 19  low    0.010       56
## 20 high    0.010       56
## 21  low    0.050       56
## 22 high    0.050       56
## 23  low    0.100       56
## 24 high    0.100       56
```

Cases 5 and 6 (with min_prpn = 0.05 and tr_start = 24) are extensively discussed in the main text.

```r
cases <- cases[5:6, ]
n_iter <- 10 # only 10 instead of 5000 for illustration
```

Running Trial Optimization Step for 10 Replications

Only selected small-scale cases are executed. One CPU core is enough.

```r
library(parallel)

num_cores <- 1

results <- mclapply(
  1:nrow(cases),
  function(i, cases, sim_choice, sim_dat, n_iter) {

    out_dir <- "output_git/"

    if (!dir.exists(out_dir)) {
      dir.create(out_dir, recursive = TRUE)
    }

    N <- sim_choice$N
    K <- sim_choice$K # cannot be changed
    d <- sim_choice$d # cannot be changed
    E <- 2 # cannot be changed

    beta_true_low <- sim_dat$beta_true
    beta_true_high <- 2 * beta_true_low

    placebo_arm <- sim_dat$placebo_arm
    weight <- sim_dat$weight
    design <- sim_choice$design
    reward_sig <- sim_choice$reward_sig

    dgp <- cases[i, "dgp"]
    min_prpn <- cases[i, "min_prpn"]
    tr_start <- cases[i, "tr_start"]

    case_str <- paste(
      "dgp", dgp,
      "min_prpn", min_prpn,
      "tr_start", tr_start,
      sep = "_"
    )
```

```r
case_str0 <- paste(
  "dgp", dgp,
  "min_prpn", 0.005,
  "tr_start", sim_choice$ate_start,
  sep = "_"
)

ts_sim <- vector(mode = "list", length = n_iter)
rand_sim <- ts_sim
rits_sim <- ts_sim
rand_mis_sim <- ts_sim
ts_mis_sim <- ts_sim
rits_mis_sim <- ts_mis_sim

if (dgp == "low") {
  beta_true <- beta_true_low
} else {
  beta_true <- beta_true_high
}

for (iter in 1:n_iter) {

  seed_ <- iter
  set.seed(seed_)

  X_true <- get_X(N = N)
  X <- X_true[, 1:2]

  ts_sim[[iter]] <- do_ts_batch(
    X = X_true,
    X_true = X_true,
    beta_true = beta_true,
    weight = weight,
    seed = seed_,
    rwd_sig = reward_sig,
    tr_start = tr_start,
    tr_batch = 5,
    tr_lag = 10,
    M = 1000,
    v = 10,
    min_prpn = min_prpn,
    asympcs = FALSE
  )

  rits_sim[[iter]] <- do_rits_batch(
    X = X_true,
    X_true = X_true,
    beta_true = beta_true,
    weight = weight,
    seed = seed_,
    rwd_sig = reward_sig,
    tr_start = tr_start,
    tr_batch = 5,
```

```r
    tr_lag = 10,
    M = 1000,
    v = 10,
    min_prpn = min_prpn,
    asympcs = FALSE
  )

  ts_mis_sim[[iter]] <- do_ts_batch(
    X = X,
    X_true = X_true,
    beta_true = beta_true,
    weight = weight,
    seed = seed_,
    rwd_sig = reward_sig,
    tr_start = tr_start,
    tr_batch = 5,
    tr_lag = 10,
    M = 1000,
    v = 10,
    min_prpn = min_prpn,
    asympcs = FALSE
  )

  rits_mis_sim[[iter]] <- do_rits_batch(
    X = X,
    X_true = X_true,
    beta_true = beta_true,
    weight = weight,
    seed = seed_,
    rwd_sig = reward_sig,
    tr_start = tr_start,
    tr_batch = 5,
    tr_lag = 10,
    M = 1000,
    v = 10,
    min_prpn = min_prpn,
    asympcs = FALSE
  )

  # Generate randomization comparator for plotting.
  # We save it using case_str0, i.e. min_prpn = 0.005 and tr_start = ate_start,
  # to match the naming convention expected by the plotting code.
  rand_sim[[iter]] <- do_rand_biv(
    X = X_true,
    X_true = X_true,
    beta_true = beta_true,
    weight = weight,
    seed = seed_,
    rwd_sig = reward_sig,
    tr_start = sim_choice$ate_start,
    asympcs = FALSE
  )
```

```r
      rand_mis_sim[[iter]] <- do_rand_biv(
        X = X,
        X_true = X_true,
        beta_true = beta_true,
        weight = weight,
        seed = seed_,
        rwd_sig = reward_sig,
        tr_start = sim_choice$ate_start,
        asympcs = FALSE
      )
    }

    saveRDS(ts_sim, paste(out_dir, "ts_sim_", case_str, ".RData", sep = ""))
    saveRDS(rits_sim, paste(out_dir, "rits_sim_", case_str, ".RData", sep = ""))
    saveRDS(ts_mis_sim, paste(out_dir, "ts_mis_sim_", case_str, ".RData", sep = ""))
    saveRDS(rits_mis_sim, paste(out_dir, "rits_mis_sim_", case_str, ".RData", sep = ""))

    saveRDS(rand_sim, paste(out_dir, "rand_sim_", case_str0, ".RData", sep = ""))
    saveRDS(rand_mis_sim, paste(out_dir, "rand_mis_sim_", case_str0, ".RData", sep = ""))

    return(NULL)
  },
  mc.cores = num_cores,
  cases = cases,
  sim_dat = sim_dat,
  sim_choice = sim_choice,
  n_iter = n_iter
)
```

Trial Evaluation

Loading Necessary Functions, Objects and Simulated Data

```r
source("code/function/main_function.R")
source("code/function/misc.R")
source("code/function/classical_ci.R")
source("code/function/asymp_cs.R")
```

```
## Loading required package: drconfseq

## Loading required package: SuperLearner

## Loading required package: nnls

## Loading required package: gam

## Loading required package: splines

## Loading required package: foreach

## Loaded gam 1.22-5

## Super Learner

## Version: 2.0-29

## Package created on 2024-02-06
```

```r
batch <- 1
```

Adding AsympCS and T-Test CI in Output Files from Trial Optimization Step

```r
library(parallel)

num_cores <- 1

results <- mclapply(
  1:nrow(cases),
  function(i, cases, sim_choice) {

    out_dir <- "output_git/"

    if (!dir.exists(out_dir)) {
      dir.create(out_dir, recursive = TRUE)
    }

    dgp <- cases[i, "dgp"]
    min_prpn <- cases[i, "min_prpn"]
    tr_start <- cases[i, "tr_start"]

    case_str <- paste(
      "dgp", dgp,
      "min_prpn", min_prpn,
      "tr_start", tr_start,
      sep = "_"
    )

    case_str0 <- paste(
      "dgp", dgp,
      "min_prpn", 0.005,
      "tr_start", sim_choice$ate_start,
      sep = "_"
    )

    file_names <- c(
      paste(out_dir, "ts_sim_", case_str, ".RData", sep = ""),
      paste(out_dir, "rits_sim_", case_str, ".RData", sep = ""),
      paste(out_dir, "ts_mis_sim_", case_str, ".RData", sep = ""),
      paste(out_dir, "rits_mis_sim_", case_str, ".RData", sep = ""),
      paste(out_dir, "rand_sim_", case_str0, ".RData", sep = ""),
      paste(out_dir, "rand_mis_sim_", case_str0, ".RData", sep = "")
    )

    for (file_name in file_names) {

      if (file.exists(file_name)) {

        out_sim <- readRDS(file_name)

        out_sim <- add_asympcs_sim(
          out_list = out_sim,
```

```r
          ate_start = 24,
          batch = batch,
          placebo_arm = 1,
          alpha = 0.05,
          first_peek = 50,
          n_cores = 1,
          force_compute = TRUE,
          learner = "main_ridge"
        )

        out_sim <- add_standard_ci(
          out = out_sim,
          ate_start = 24,
          placebo_arm = 1,
          force_compute = TRUE
        )

        print(file_name)
        print(length(out_sim))

        saveRDS(out_sim, file_name)

      } else {
        warning("File not found and skipped: ", file_name)
      }
    }

    return(NULL)
  },
  mc.cores = num_cores,
  cases = cases,
  sim_choice = sim_choice
)
```

```
## [1] "output_git/ts_sim_dgp_low_min_prpn_0.05_tr_start_24.RData"
## [1] 10
## [1] "output_git/rits_sim_dgp_low_min_prpn_0.05_tr_start_24.RData"
## [1] 10
## [1] "output_git/ts_mis_sim_dgp_low_min_prpn_0.05_tr_start_24.RData"
## [1] 10
## [1] "output_git/rits_mis_sim_dgp_low_min_prpn_0.05_tr_start_24.RData"
## [1] 10
## [1] "output_git/rand_sim_dgp_low_min_prpn_0.005_tr_start_24.RData"
## [1] 10
## [1] "output_git/rand_mis_sim_dgp_low_min_prpn_0.005_tr_start_24.RData"
## [1] 10
## [1] "output_git/ts_sim_dgp_high_min_prpn_0.05_tr_start_24.RData"
## [1] 10
## [1] "output_git/rits_sim_dgp_high_min_prpn_0.05_tr_start_24.RData"
## [1] 10
## [1] "output_git/ts_mis_sim_dgp_high_min_prpn_0.05_tr_start_24.RData"
## [1] 10
## [1] "output_git/rits_mis_sim_dgp_high_min_prpn_0.05_tr_start_24.RData"
## [1] 10
```

```
## [1] "output_git/rand_sim_dgp_high_min_prpn_0.005_tr_start_24.RData"
## [1] 10
## [1] "output_git/rand_mis_sim_dgp_high_min_prpn_0.005_tr_start_24.RData"
## [1] 10
```

Generate All Plots

Main Text Example Plot

This uses the small-scale example files generated in `output_git/`.

```r
required_pkgs <- c("patchwork", "reshape2", "ggh4x", "colorspace")

missing_pkgs <- required_pkgs[!vapply(required_pkgs, requireNamespace, logical(1), quietly = TRUE)]

if (length(missing_pkgs) > 0) {
  stop(
    "Please install the following packages before running the plotting chunk: ",
    paste(missing_pkgs, collapse = ", ")
  )
}

library(patchwork)
library(reshape2)
library(ggh4x)
```

```
## Loading required package: ggplot2
```

```r
library(colorspace)

source("code/visualization/main_viz_example.R")
```

```
##
## Attaching package: 'tidyr'

## The following object is masked from 'package:reshape2':
##
##     smiths

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

The generated example plot object is:

- `metric_alloc_plot_example`

The saved plot file is:

- `plot/metric_alloc_plot_example.jpg`

Web Appendix

The full appendix plotting script expects the full author simulation outputs, so it is not run for this small-scale example.

```
source("code/visualization/appendix_viz.R")
```

Generate All Tables

Main Text

The full table script expects the full author simulation outputs, so it is not run for this small-scale example.

```
source("code/visualization/main_tab.R")
est_err_tab <- rbind(est_err_tab_high, est_err_tab_low)
```

Web Appendix

The full appendix table script expects the full author simulation outputs, so it is not run for this small-scale example.

```
source("code/visualization/appendix_tab.R")
table2 <- rbind(est_err_all_mis_tab[[6]]$tab, est_err_all_mis_tab[[5]]$tab)
```